# Terraform and GitHub Automation with Claude: Research and Content Plan

Network and Infrastructure Automation · 3 days (8 hours per day) · Beginner to intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.


- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References


## 1. Where this course came from

**Client input.**

> Client use case 11, "Terraform and GitHub Automation": "Terraform scripting step by step and common mistakes. How to use GitHub repository." The client's stated aim for all use cases: "Our Aim is to do automation using Claude for all these content", with titles of the form "Automation of <> using Claude". Related use cases name the estate: F5, NetScaler, Radware and Anaya LB load balancers and Cisco ACI.

Network and load balancer teams are adopting infrastructure as code, but most start from GUI and CLI habits. The client asked for Terraform taught step by step, the common mistakes, and how to use a GitHub repository. Those mistakes are well documented and costly: unpinned providers, secrets that end up in state and plan files, state committed to Git, count where for_each belonged, renames that destroy live objects, and targeted applies that leave drift [10] [12] [13] [16] [19].

This course teaches Terraform for the estate the client runs: F5 BIG-IP, NetScaler and Cisco ACI through their published providers, and Radware Alteon and other load balancers through Python wrappers in the same pipeline [38] [41] [43] [55]. GitHub is taught as the control plane for change: protected branches, CODEOWNERS, plan on pull request, apply on merge, and environment approvals [5] [6] [7] [27].

Claude is added where it helps and where it is safe: drafting HCL and workflows, explaining plans in plain English, and reviewing plan JSON with a structured risk verdict. Claude never holds apply. Permission rules, tool restriction, policy checks and human approval keep it that way, and every change leaves an audit record [11] [48] [51] [52].

## 2. How the outline was shaped

| Decision | Rationale |
|---|---|
| Teach Terraform from zero before adding automation. | The client input asks for 'Terraform scripting step by step', and the audience prerequisites say no Terraform is required. Day 1 builds a real module, import and remote state before any pipeline [3] [17] [20]. |
| A whole module on common mistakes, reproduced in the lab. | The client named 'common mistakes'. Each mistake is reproduced, its symptom shown in plan output and a check wired to catch it, which sticks better than a list [4] [12] [13] [26] [35]. |
| GitHub as the change process, not just storage. | The client asked 'how to use GitHub repository'. For infrastructure that means protection, ownership, required checks, environments and secrets, not only commits [5] [6] [27] [34]. |
| Claude reviews and explains, but never applies. | An agent with apply rights is a production risk. Deny rules, tools=[] and an MCP server with no write tools keep Claude out of apply; prompt text alone is not enforcement [48] [49] [51]. |
| Structured verdicts from plan JSON. | Plan JSON gives exact actions and replace_paths, so the verdict can be tested and compared with policy results rather than trusted as free text [11] [52]. |
| Cover F5, NetScaler, ACI with providers and Radware or Anaya LB through wrappers. | These are the platforms in the client's use cases; where no maintained provider is used, a Python wrapper keeps the same gate and audit [38] [41] [43] [55]. |

## 3. Audience and prerequisites

| Role | What they need | Starting point |
|---|---|---|
| Load balancer engineer (F5, NetScaler, Radware) | Manage VIPs, pools and certificates as code with safe review. | Strong GUI and CLI skills; little or no Terraform or Git. |
| Data center and ACI engineer | Manage tenants and bridge domains through reviewed code. | Knows ACI concepts; new to Terraform. |
| Network operations engineer | Understand plans and approve changes confidently. | Basic Linux; reads configs daily. |
| Platform or DevOps engineer | Pipelines, policy and governance for network IaC. | Knows Git and CI; may be new to network providers. |
| Team lead or change manager | An approval process and audit trail that holds up. | Approves changes today; wants clear evidence. |

### Prerequisites from the outline

- Basic Linux command line and text editing
- Basic networking and load balancer concepts (VIP, pool, certificate)
- Git basics are helpful, not required
- No prior Terraform or Python experience required

### Checks before enrolment

- Can you edit a text file on Linux and run commands in a terminal?
- Can you explain what a VIP, a pool and a pool member are?
- Do you know what a TLS certificate is used for on a load balancer?
- Have you used Git at all (clone, commit)? Helpful, not required.
- Do you have a GitHub account you can add to the course organization?

### Course outcomes

- Write, plan and apply Terraform configurations step by step with providers, variables, modules, remote state and locking
- Recognise and fix the common Terraform mistakes: unpinned providers, secrets in code, state in Git, count versus for_each, targeted applies and drift
- Run a GitHub repository for infrastructure code with branches, pull requests, CODEOWNERS, branch protection and GitHub Actions plan-on-PR and apply-on-merge
- Set up Claude Code, the Claude Agent SDK and an MCP server so Claude can draft, plan and review Terraform without ever running apply itself
- Build a plan-review agent that reads terraform show -json output and posts a structured risk verdict to the pull request
- Enforce guardrails, environment approvals and an audit trail for every plan and apply across F5, NetScaler, Cisco ACI and other load balancers (Anaya LB)

## 4. Lab environment

Each learner gets a Linux VM with Terraform, Git, the GitHub CLI, Python and Claude Code; a seat in a shared GitHub organization; lab F5 BIG-IP VE and NetScaler VPX Express instances; and access to an APIC simulator. Remote state lives in a lab backend with locking. Nothing in the lab can reach a production device.

### Components

| Component | Version or edition | Purpose | How to obtain | Notes |
|---|---|---|---|---|
| Learner VM | Ubuntu 24.04 LTS with Terraform CLI 1.x (latest at delivery, pinned in the lab guide), Git, GitHub CLI, Python 3.12 | Authoring, local plans and Claude Code. | Course image from the training team. | HashiCorp docs currently list v1.17.x as the latest documented release; pin one version for the class [20]. |
| Terraform providers | F5Networks/bigip 1.29.0, citrix/citrixadc 3.0.3, CiscoDevNet/aci 2.20.0 at time of writing | Device management. | Downloaded from the Terraform Registry on init [47]. | Pin with ~> and commit .terraform.lock.hcl [15] [16]. |
| GitHub organization | GitHub Enterprise Cloud (or public lab repositories on Team plan) | Repositories, Actions, environments with required reviewers. | Course organization created by the training team. | Required reviewers on private repositories need Enterprise Cloud [27]. |
| Self-hosted Actions runner | Current GitHub Actions runner on the lab network | Lets workflows reach lab devices. | Registered to the course organization by the lab team. | GITHUB_TOKEN lifetime on self-hosted runners is at most 24 hours [32]. |
| F5 BIG-IP Virtual Edition | BIG-IP VE 17.x LTM | F5 provider labs. | 30-day trial via MyF5 or lab license [57]. | AS3 extension installed for the optional AS3 demo [8]. |
| NetScaler VPX Express | NetScaler VPX 14.1 Express | citrixadc provider labs. | Free download, no license file [58]. | No vendor support cases on Express. |
| Cisco APIC simulator | DevNet ACI Simulator sandbox (reserved) or ACI Simulator VM | ACI provider labs. | Cisco DevNet sandbox reservation, up to 5 days [56]. | Use a reserved sandbox; the Always-On instance is shared. |
| Radware Alteon | Alteon VA 32.x or recorded responses | Python wrapper labs. | Partner evaluation or mock server with recorded responses [55]. | Anaya LB is taught with a mock API following the same wrapper contract. |
| Policy and lint tools | Conftest, TFLint, pre-commit (pinned) | Guardrails and mistake catching. | Installed on the VM image [26] [36] [37]. | Same versions in CI. |

| | | | | |
|---|---|---|---|---|
| Claude Code and Claude Agent SDK (Python) | Current releases at delivery | Drafting, explaining and plan review. | Organization API key or seats. | Settings and hooks provided in the repository template [48] [53]. |

### Fixtures and sample data

| Fixture | Contents | Used in |
|---|---|---|
| Course repository template | Layout, .gitignore, CODEOWNERS, workflows, policy pack, CLAUDE.md and settings. | GitHub repository fundamentals for infrastructure code; GitHub Actions pipelines: plan on pull request, apply on merge |
| Mistakes branch | Five seeded mistakes with expected symptoms. | Common Terraform mistakes and how to catch them early |
| Fixture plan pack | Plan JSON for delete, replace, certificate change, out-of-scope and no-op cases. | Claude as plan reviewer: Agent SDK, MCP and structured verdicts; Guardrails, approvals and audit for Terraform and GitHub automation |
| Legacy objects | Pre-built F5 pool and ACI tenant for import labs. | Terraform scripting step by step: init, plan, apply, state; Terraform for F5, NetScaler, Cisco ACI and other load balancers |
| Capstone change ticket | Fictional CHG ticket with requirements and a seeded rename problem. | Capstone: end-to-end change through the Claude-reviewed pipeline |

### Lab safety rules

- Lab devices are isolated; no route to production.
- Claude has no apply, destroy, import or state tools and Bash apply is denied.
- Device credentials live in environment secrets released only after approval.
- All names, tickets and addresses are fictional.
- Instructors hold the only admin accounts on lab devices and the organization.

## 5. Module-by-module content plan

### Day 1

> **Module 1: Infrastructure as code for network and load balancer teams**

**Goal.** Show network and load balancer engineers why declarative Terraform beats click-ops and ad hoc scripts, and get every learner to a first successful plan.

**Outline topics.**

GenmoTech TRAINING For more info - https://genmo.com

- Why Terraform for F5, NetScaler, Cisco ACI and cloud: declarative config versus click-ops and ad hoc scripts
- HCL building blocks: providers, resources, data sources, variables, outputs and the state file
- The course pattern: Claude plans and explains, Terraform and pipelines execute, a human approves, everything is logged
- Lab environment tour: GitHub organization, lab F5 and NetScaler, Cisco ACI simulator, Terraform CLI on Linux

**Learning objectives.**

- Learners can explain declarative configuration and the role of the state file.
- Learners can name the HCL building blocks: providers, resources, data sources, variables, outputs and modules.
- Learners can describe the course pattern of Claude planning and reviewing, pipelines executing and humans approving.
- Learners can run terraform init and plan against a sample F5 pool configuration.

**Why Terraform for network teams**

- Click-ops leaves no review trail; scripts are imperative and rarely idempotent; Terraform compares desired configuration with recorded state and real objects and plans only the difference [1].
- Vendor providers exist for the estate in scope: F5Networks/bigip, citrix/citrixadc and CiscoDevNet/aci, all on the public registry [47].
- A plan is a reviewable artifact: + create, - destroy, ~ update in place, -/+ replace [10].
- The cost is discipline: state must be protected, providers pinned and every change go through review.

**HCL building blocks**

- terraform { required_providers { bigip = { source = "F5Networks/bigip", version = "~> 1.29" } } } pins the provider [15] [47].
- provider "bigip" {} reads BIGIP_HOST, BIGIP_USER and BIGIP_PASSWORD from the environment, so no credentials live in code [38].
- resource blocks declare objects; data sources read existing ones; variables and outputs form the interface; modules group resources [3].
- State is JSON that maps addresses such as bigip_ltm_pool.web to real objects; never edit it by hand or commit it [1].

**The course pattern**

- Claude drafts HCL, explains plans and reviews pull requests; it never runs apply.
- Terraform runs in GitHub Actions; apply happens only on merge and after environment approval [7] [27].
- Every change has an audit record: request, plan hash, Claude verdict, approver and run URL.

- Policy and tests catch what review misses.

**Lab environment tour**

- GitHub organization with a team repository template.
- Lab F5 BIG-IP VE, NetScaler VPX Express and an APIC simulator [57] [58] [56].
- Terraform CLI on a Linux VM; providers downloaded on init and locked in .terraform.lock.hcl [16].
- All devices are lab-only with fictional names.

**Instructor demo.**

1. Create a pool in the F5 GUI, then show the same pool as 12 lines of HCL.
2. Run terraform init and show the provider download and lock file [16].
3. Run terraform plan and read the symbols aloud [10].
4. Change the pool's load balancing mode in the GUI and re-run plan to show detected drift.
5. Show where state lives after a local apply and why that is a problem for teams [1].

**Hands-on lab.** Install Terraform, clone the course repository and run terraform init and plan against a sample F5 BIG-IP pool configuration.

1. Install the Terraform CLI version given in the lab guide and confirm terraform version.
2. Clone the course repository and open examples/f5-pool.
3. Export BIGIP_HOST, BIGIP_USER and BIGIP_PASSWORD for the lab device [38].
4. Run terraform init and inspect .terraform.lock.hcl [16].
5. Run terraform fmt -check and terraform validate [4].
6. Run terraform plan and write down each planned action.
7. Change one argument and plan again; note the symbol change.
8. Do not apply; the apply happens through the pipeline later.

**Lab success criteria.**

- terraform init completes and the lock file exists.
- A plan shows the expected pool, node and member creations.
- No credentials are written to any .tf or .tfvars file.
- The learner can explain each plan symbol.

**Deliverable.** Working Terraform workstation and first successful plan.

**Common pitfalls.**

- Putting device passwords in provider blocks.
- Running apply locally 'just to test' and creating unmanaged state.
- Ignoring the lock file and getting different provider versions per laptop [16].
- Treating a plan as a guarantee rather than a prediction from current state.

**Knowledge check.**

1. What does -/+ mean in a plan? Answer: Destroy then create a replacement [10].
2. Why should state not be committed to Git? Answer: It can contain sensitive values and must not be hand-edited or versioned there; use a remote backend [1] [19].
3. How does the F5 provider get credentials without code? Answer: From BIGIP_HOST, BIGIP_USER and BIGIP_PASSWORD environment variables [38].

**Slides and assets to prepare.**

- Click-ops versus scripts versus Terraform comparison slide.
- HCL anatomy poster with an F5 pool example.
- Course pattern diagram: Claude, pipeline, approver, audit.
- Lab access sheet.

**Sources for this module.** 1, 3, 4, 7, 10, 15, 16, 19, 27, 38, 47, 56, 57, 58

## Module 2: Setting up Claude Code, the Agent SDK and MCP for Terraform work

**Goal.** Configure Claude so it can format, validate, plan and explain Terraform but has no route to apply.

**Outline topics.**

- Claude Code in the repository: CLAUDE.md conventions, allowed commands, and why apply is never on the allowed list
- Claude Agent SDK basics in Python: a session, tool use, structured JSON output with a schema
- An MCP server that exposes terraform fmt, validate, plan and show -json as read-only tools
- Prompting Claude to explain a plan in plain English and cite the resource addresses it refers to

**Learning objectives.**

- Learners can write CLAUDE.md conventions and permission rules for a Terraform repository.
- Learners can explain why deny rules and tool removal, not prompt text, keep apply out of reach.
- Learners can build an MCP server exposing fmt, validate, plan and show -json as tools.
- Learners can use the Agent SDK to get a structured plan explanation that cites resource addresses.

**Claude Code in the repository**

- CLAUDE.md at ./CLAUDE.md or ./.claude/CLAUDE.md records layout, naming, module conventions and the rule that apply is never run by Claude [49].
- CLAUDE.md is context, not enforcement; enforcement is permission rules and hooks [49].
- Deny rules: Bash(terraform apply *), Bash(terraform destroy *), Bash(terraform import *), Bash(terraform state *) and Bash(terraform force-unlock *); deny wins over any allow [48].

- Bash rules match command text, so a determined path around them exists (for example a script that calls terraform); for the review agent, remove Bash entirely [48].

**Agent SDK basics**

- A query() session with ClaudeAgentOptions(tools=[], mcp_servers={...}, allowed_tools=['mcp__tf__plan', ...]) gives Claude only Terraform tools [51].
- Structured output with output_format and a Pydantic schema returns validated JSON in structured_output [52].
- PreToolUse hooks can deny any tool call whose arguments point outside the repository [53].
- PostToolUse hooks log every call for the audit record [53].

**A Terraform MCP server**

- Tools: fmt_check(dir), validate(dir), plan(dir, var_file) that always writes -out=plan.tfplan, and show_json(plan_file) [10] [23].
- Each tool runs terraform with -input=false and TF_IN_AUTOMATION=1 [24].
- No apply, destroy, import or state tools exist on the server at all.
- Validate inputs: the directory must be under envs/ or modules/, var files must match an allow list [54].
- Register with claude mcp add --scope project tf -- python -m tf_mcp [50].

**Explaining a plan**

- Prompt Claude to explain each resource change by address, for example bigip_ltm_virtual_server.vs_web_443, and the action list from show -json [11].
- Ask Claude to list unknown values (after_unknown) instead of guessing them [11].
- Never pass sensitive values: show -json prints sensitive state values in plain text, so the tool strips fields marked in before_sensitive and after_sensitive [23] [11].
- Claude's explanation ends with a statement of what it cannot do, including apply.

**Instructor demo.**

1. Show .claude/settings.json with deny rules for apply and destroy [48].
2. Ask Claude Code to run terraform apply and show the denial.
3. Register the MCP server and list its tools [50].
4. Run the Agent SDK explainer on a saved plan and show structured output with addresses [52].
5. Show the redaction step removing a sensitive attribute before Claude sees the plan [23].

**Hands-on lab.** Configure Claude Code for the repo, start the Terraform MCP server, and ask Claude to explain a plan without being able to apply it.

1. Write CLAUDE.md with repository conventions.
2. Create .claude/settings.json with the deny rules and allowed MCP tools.
3. Implement the MCP server with fmt_check, validate, plan and show_json.

4. Add sensitive-value redaction to show_json.
5. Register the server at project scope.
6. Ask Claude Code to plan the F5 example and explain it.
7. Ask Claude to apply and record the refusal.
8. Run the Agent SDK explainer and save the structured output.

**Lab success criteria.**

- Claude produces a plan explanation citing every changed address.
- Attempts to apply or destroy are refused and logged.
- No sensitive value appears in any Claude input or output.
- The MCP server has no tool that can change infrastructure.

**Deliverable.** Claude Code plus MCP setup with a read-only Terraform tool set.

**Common pitfalls.**

- Relying on a CLAUDE.md line saying 'never apply' [49].
- Allowing Bash(terraform *) and trying to carve out apply with a narrower rule; it does not work that way in reverse [48].
- Feeding raw show -json with sensitive values to the model [23].
- Plan tools that do not write -out, so the reviewed plan is lost [10].

**Knowledge check.**

1. Which comes first in Claude Code rule evaluation: allow or deny? Answer: Deny, then ask, then allow [48].
2. What does terraform show -json do with sensitive state values? Answer: Prints them in plain text, so they must be redacted [23].
3. How do you make sure the SDK agent has no shell? Answer: Pass tools=[] and expose only MCP Terraform tools [51].

**Slides and assets to prepare.**

- CLAUDE.md and settings.json templates for Terraform repositories.
- Terraform MCP server skeleton.
- Plan explanation prompt and schema.
- Slide: what Claude can and cannot do in this course.

**Sources for this module.** 10, 11, 23, 24, 48, 49, 50, 51, 52, 53, 54

## Module 3: Terraform scripting step by step: init, plan, apply, state

**Goal.** Teach the full Terraform lifecycle hands-on, from first module to import and a locked remote backend.

**Outline topics.**

Gennoor Tech INNOVATE BUILD For more info - https://gennoor.com

- The lifecycle: init, fmt, validate, plan, apply, destroy; reading a plan symbol by symbol (+, -, ~, -/+)
- Variables, tfvars files, locals, outputs and the first reusable module for an F5 virtual server and pool
- terraform import for a virtual server that already exists; terraform state list, show, mv and rm
- Remote state with locking, workspaces for dev and prod, and pinned provider versions in required_providers

**Learning objectives.**

- Learners can run init, fmt, validate, plan, apply and destroy and read plan output symbol by symbol.
- Learners can build a reusable module for an F5 virtual server and pool with variables and outputs.
- Learners can import an existing object and use state list, show, mv and rm safely.
- Learners can configure a locked remote backend, pinned providers and separate dev and prod state.

**The lifecycle and plan reading**

- init downloads providers and configures the backend; fmt and validate check syntax; plan predicts; apply executes; destroy removes [4] [10].
- Plan symbols: + create, - destroy, ~ update in place, -/+ destroy then create replacement [10].
- '# forces replacement' next to an attribute tells you why a resource will be replaced.
- terraform plan -out=tfplan then terraform apply tfplan applies exactly what was reviewed [10] [24].

**Variables, locals, outputs and a first module**

- modules/f5-vserver/ has variables name, destination, port, members (map) and monitors; outputs vs_name and pool_name [3].
- bigip_ltm_virtual_server requires name, destination and port, optionally pool [39].
- Pool members use bigip_ltm_pool_attachment with node such as 10.20.1.15:443 [40].
- tfvars files per environment hold non-secret values; secrets come from TF_VAR_ environment variables with sensitive = true on the variable [18].
- Locals compute names consistently, for example local.vs_name = "vs-${var.app}-${var.port}".

**Import and state commands**

- terraform import bigip_ltm_pool.web /Common/web-pool imports an existing pool by full path [17] [39].
- An import block in configuration lets the import go through plan and review, which is better for teams [17].
- terraform state list and state show inspect; state mv SOURCE DESTINATION re-addresses; state rm stops tracking without deleting [22].

- Prefer moved blocks over state mv so the change is reviewed in a pull request [12] [22].

**Remote state, locking, workspaces and pinning**

- Only one backend block per configuration; backend credentials go in environment variables because backend config is written in plain text to .terraform and plan files [2].
- Choose a backend that supports locking; Terraform then locks automatically on any write operation [20].
- force-unlock only releases your own stuck lock; never break a colleague's lock [20].
- Workspaces share credentials and code; for prod with different credentials and approvals, use separate directories envs/dev and envs/prod [21].
- Pin with version = "~> 1.29" and commit .terraform.lock.hcl; upgrade only with terraform init -upgrade in a reviewed change [15] [16].

**Instructor demo.**

1. Ask Claude Code to draft the f5-vserver module from a spec; review the diff and fix one naming issue.
2. Plan, save with -out and apply the saved plan in the lab dev environment [10].
3. Import an existing pool created in the GUI using an import block [17].
4. Rename a resource and show the destroy and create, then fix it with a moved block [12].
5. Migrate local state to the remote backend with terraform init -migrate-state and show the lock during a plan [20].

**Hands-on lab.** Build a module for an F5 virtual server with Claude drafting the HCL, import an existing pool, and move state to a locked remote backend.

1. Ask Claude Code to draft modules/f5-vserver from the spec sheet and review every line.
2. Write envs/dev/main.tf calling the module with dev tfvars.
3. Run fmt, validate and plan; save the plan with -out.
4. Apply the saved plan to the lab F5 (dev only, instructor-approved).
5. Write an import block for the pre-built legacy pool and plan the import.
6. Configure the remote backend with locking and migrate state.
7. Run two plans at once from two terminals and observe the lock.
8. Commit code, lock file and tfvars without secrets.

**Lab success criteria.**

- The module is reusable with no hard-coded addresses.
- The legacy pool is in state with a clean plan afterwards.
- State lives only in the locked remote backend.
- The second concurrent plan waits or fails on the lock.

**Deliverable.** Reusable virtual server module with remote, locked state.

**Common pitfalls.**

- Using workspaces for prod and dev with the same credentials [21].
- Using terraform state mv in a terminal instead of a reviewed moved block [12].
- Forgetting that import does not write configuration; the resource block must match.
- Choosing a backend without locking [20].

**Knowledge check.**

1. How do you guarantee apply uses the plan that was reviewed? Answer: Save it with plan -out and pass the file to apply [10].
2. What does ~> 1.29 allow? Answer: 1.29 and later 1.x versions, but not 2.0 [15].
3. When is terraform force-unlock appropriate? Answer: Only to release your own lock when automatic unlocking failed [20].

**Slides and assets to prepare.**

- Plan reading worksheet with annotated output.
- F5 virtual server module reference solution.
- Backend and locking setup guide for the lab.
- State command cheat sheet.

**Sources for this module.** 2, 3, 4, 10, 12, 15, 16, 17, 18, 20, 21, 22, 24, 39, 40

## Day 2

### Module 4: Common Terraform mistakes and how to catch them early

**Goal.** Reproduce the classic Terraform mistakes in the lab, learn each symptom, and wire checks that catch them before review.

**Outline topics.**

- Unpinned provider versions, secrets hard coded in variables, and the .tfstate file committed to Git
- count versus for_each, resource recreation from a renamed block, and the moved block as the fix
- Drift left behind by terraform apply -target, manual changes on the device, and detecting drift with plan -detailed-exitcode
- Wiring the catchers: terraform fmt -check, validate, tflint, a pre-commit hook and a Claude review that names the mistake and the line

**Learning objectives.**

- Learners can name, reproduce and fix at least five common Terraform mistakes.
- Learners can choose correctly between count and for_each and refactor with moved blocks.
- Learners can detect drift with plan -detailed-exitcode and explain why -target causes it.

Gennoor Tech TRAIN · INNOVATE · BUILD For more info - https://gennoor.com

- Learners can set up fmt, validate, tflint, pre-commit and a Claude review that names the mistake and line.

**Pinning, secrets and state in Git**

- Unpinned providers: without a version constraint and lock file, a new major provider can change behavior on the next init; pin with ~> and commit the lock file [15] [16].
- Secrets in variables or tfvars end up in state and plan files even if marked sensitive in output [19] [18].
- Use TF_VAR_ environment variables from GitHub environment secrets, and sensitive = true to hide values in output [18] [34].
- A committed terraform.tfstate exposes everything in it; .gitignore must cover *.tfstate, *.tfstate.*, .terraform/ and *.tfvars [35] [1].
- The GitHub Terraform.gitignore template does not ignore plan files by default; add *.tfplan yourself [35].

**count, for_each and renames**

- count gives index addresses such as bigip_ltm_pool_attachment.m[0]; removing the first member shifts every index and recreates attachments [13].
- Use count only for nearly identical instances and for_each when instances have distinct identities, such as pool members keyed by address [13] [14].
- for_each keys must be known at plan time [14].
- Renaming a resource block plans destroy and create; a moved block maps old to new address without destroy [12].
- Moving from count to for_each needs moved blocks for each index-to-key pair.

**Drift and targeted applies**

- -target is for exceptional circumstances; it leaves the rest of the configuration unapplied and the next full plan shows surprises [10].
- Manual changes on the device cause drift that the next plan reverses, possibly during an unrelated change.
- terraform plan -detailed-exitcode returns 2 when changes exist, so a scheduled job can flag drift [10].
- terraform plan -refresh-only shows what changed outside Terraform without proposing configuration changes [10].

**Wiring the catchers**

- terraform fmt -check -recursive fails on unformatted files [4].
- terraform validate catches syntax and type errors; tflint adds rules such as unused declarations and deprecated syntax, configured in .tflint.hcl with tflint --init [26].
- pre-commit runs fmt, validate and tflint on every commit after pre-commit install [37].

- A Claude review step reads the diff and plan JSON and must name the mistake, the file and the line, with a fix.
- The same checks run in CI so a skipped local hook does not matter.

**Instructor demo.**

1. Remove the version constraint, run init -upgrade against a newer test provider and show the plan change [16].
2. Commit a tfstate file and show what is inside it.
3. Remove the first member from a count-based list and show the recreate cascade [13].
4. Fix it with for_each and moved blocks [12] [14].
5. Run -target on one resource, then a full plan, and show the leftover changes [10].
6. Run pre-commit and the Claude review on a branch with all five mistakes.

**Hands-on lab.** Reproduce five classic mistakes in the lab repo, watch each one surface in plan or review, and fix them one by one.

1. Check out the mistakes branch with an unpinned provider, a hard-coded password, a committed tfstate, a count-based member list and a renamed resource.
2. Install pre-commit and the hook config; run pre-commit run --all-files [37].
3. Run tflint and record its findings [26].
4. Run the Claude review and compare its findings with tflint.
5. Fix the provider pin and commit the lock file.
6. Move the password to a TF_VAR_ variable marked sensitive.
7. Remove the committed state file, fix .gitignore, and note that history still holds the old copy until it is purged.
8. Convert count to for_each and add moved blocks; rename with a moved block; confirm a clean plan.

**Lab success criteria.**

- All five mistakes are fixed and plan shows no unintended destroy.
- pre-commit and CI checks pass.
- The mistake catalogue has symptom, cause, fix and the check that caught it for each mistake.
- No secret remains in the repository working tree.

**Deliverable.** Mistake catalogue with reproduction and fix for each, plus pre-commit checks.

**Common pitfalls.**

- Believing sensitive = true keeps secrets out of state [19].
- Deleting a committed state file in a new commit and assuming it is gone from history.
- Using -target as a routine way to 'just change one thing' [10].
- Relying only on local hooks that developers can skip.

**Knowledge check.**

1. Why does removing the first element of a count list recreate other resources? Answer: Instances are addressed by index, so every later index shifts [13].
2. What exit code does plan -detailed-exitcode return when changes are present? Answer: 2 [10].
3. Does the GitHub Terraform.gitignore template ignore plan files? Answer: No, that line is commented out; add *.tfplan [35].
4. What does a moved block do? Answer: Tells Terraform an object moved address so it is renamed in state, not destroyed [12].

**Slides and assets to prepare.**

- Mistake catalogue template.
- Mistakes branch in the lab repository.
- .pre-commit-config.yaml and .tflint.hcl.
- Before and after plan screenshots for each mistake.

**Sources for this module.** 1, 4, 10, 12, 13, 14, 15, 16, 18, 19, 26, 34, 35, 37

## Module 5: GitHub repository fundamentals for infrastructure code

**Goal.** Set up a protected team repository with a clear layout, review ownership and safe secret handling.

**Outline topics.**

- Repository layout for Terraform: environments, modules, .gitignore for state, plans and credentials
- Branches, commits, pull requests, reviews and merge strategies for a network team
- CODEOWNERS, branch protection rules, required reviewers and status checks
- Secrets and variables in GitHub: repository secrets, environment secrets, OIDC to cloud and Vault, and short-lived device credentials fetched at run time

**Learning objectives.**

- Learners can lay out a Terraform repository by environment and module with a correct .gitignore.
- Learners can use branches, commits and pull requests with a merge strategy suited to a network team.
- Learners can configure CODEOWNERS, branch protection, required reviews and status checks.
- Learners can choose between repository secrets, environment secrets and OIDC for credentials.

**Repository layout**

- modules/f5-vserver, modules/netscaler-vserver, modules/aci-bd; envs/dev and envs/prod; policy/; .github/workflows/.

- .gitignore covers .terraform/, *.tfstate, *.tfstate.*, *.tfvars with secrets, crash logs and *.tfplan [35].
- Commit .terraform.lock.hcl for every root module [16].
- README per environment: backend, approvers and how to raise a change.

**Branches, commits and pull requests**

- One change per branch: feature/add-vs-web-443; small commits with meaningful messages.
- Open the pull request with gh pr create --base main and a template that links the change ticket.
- Squash merge keeps one commit per change, making revert simple.
- Reviews focus on the plan comment, not just the diff.

**CODEOWNERS and branch protection**

- CODEOWNERS in .github/ maps modules/f5-* to @netops/f5-team and modules/aci-* to @netops/aci-team; owners need write access [6].
- CODEOWNERS is read from the base branch, so changes to it are themselves reviewed [6].
- Protect main: require pull requests, required reviews from code owners, required status checks (fmt, validate, tflint, plan, policy) and block force pushes [5].
- Enable 'Do not allow bypassing the above settings' so admins follow the same rules [5].
- Only one branch protection rule applies to a branch; rulesets can layer, which matters for larger organizations [5].

**Secrets and credentials**

- Repository secrets for low-risk values; environment secrets for device credentials, released only after environment approval [34].
- OIDC removes long-lived cloud credentials from GitHub by exchanging a short-lived token [28].
- For on-premises devices, fetch short-lived credentials from Vault at run time using the job's OIDC identity.
- Plan jobs use read-only device accounts; only apply jobs get write credentials.

**Instructor demo.**

1. Create the repository from the template and walk the layout.
2. Add CODEOWNERS and show automatic review requests on a pull request [6].
3. Configure branch protection and show a blocked direct push to main [5].
4. Create dev and prod environments with environment secrets [34].
5. Raise a pull request with gh pr create and walk the review.

**Hands-on lab.** Create the team repository, protect main, add CODEOWNERS for the F5 and ACI folders, and raise your first pull request from a feature branch.

1. Create the team repository from the course template in the lab organization.
2. Review and extend .gitignore [35].

3. Add CODEOWNERS for modules/f5-* and modules/aci-* [6].
4. Protect main with required reviews, code owner review and required checks [5].
5. Create dev and prod environments and add environment secrets [34].
6. Create a feature branch, add a variable to the F5 module and push.
7. Open a pull request with the template and request review.
8. Try to push directly to main and record the refusal.

**Lab success criteria.**

- Direct pushes to main are blocked for everyone, including admins.
- Code owners are requested automatically on the pull request.
- No secret value exists in the repository.
- The pull request links a change ticket.

**Deliverable.** Protected GitHub repository with pull request workflow.

**Common pitfalls.**

- Leaving admins able to bypass protection [5].
- CODEOWNERS entries for teams without write access [6].
- Putting prod device credentials in repository secrets visible to every workflow.
- Large multi-change pull requests that nobody can review.

**Knowledge check.**

1. Which copy of CODEOWNERS applies to a pull request? Answer: The one on the base branch [6].
2. When can a job read environment secrets that have required reviewers? Answer: Only after approval is granted [34].
3. What does OIDC remove from the pipeline? Answer: Long-lived cloud credentials stored as GitHub secrets [28].

**Slides and assets to prepare.**

- Repository template with layout and .gitignore.
- CODEOWNERS example.
- Branch protection settings checklist.
- Pull request template.

**Sources for this module.** 5, 6, 16, 28, 34, 35

## Module 6: GitHub Actions pipelines: plan on pull request, apply on merge

**Goal.** Build the two workflows that make every change visible as a plan and apply only the reviewed plan after approval.

Gemnoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gemnoor.com

**Outline topics.**

- Workflow anatomy: triggers, jobs, hashicorp/setup-terraform, caching and concurrency groups to avoid parallel applies
- Plan on every pull request with the plan output posted as a PR comment; fmt, validate and tflint as required checks
- Apply only on merge to main, scoped to a GitHub environment with required reviewers
- Saving the plan file as an artifact so the apply uses exactly the plan that was reviewed

**Learning objectives.**

- Learners can write workflows with triggers, jobs, setup-terraform and concurrency groups.
- Learners can post the plan as a pull request comment and run fmt, validate and tflint as required checks.
- Learners can scope apply to a protected environment with required reviewers.
- Learners can carry the reviewed plan file to the apply job safely.

**Workflow anatomy**

- on: pull_request for plan; on: push to main for apply, as in HashiCorp's reference workflow [7].
- hashicorp/setup-terraform installs a pinned terraform_version; its wrapper exposes stdout, stderr and exitcode [25].
- concurrency: group: tf-${{ github.workflow }}-prod with cancel-in-progress: false keeps applies serial; never cancel a running apply [29].
- Cache the provider plugin directory keyed on .terraform.lock.hcl to speed up init.

**Plan on every pull request**

- Steps: fmt -check -recursive, init -input=false, validate, tflint, plan -out=tfplan -input=false [4] [24] [26].
- Post the plan as a comment with actions/github-script; trim to stay under GitHub's 65535-character comment limit and update the existing bot comment [25].
- Set permissions: contents: read, pull-requests: write for the plan job [31].
- Make the checks required in branch protection [5].

**Apply only on merge, behind an environment**

- The apply job sets environment: prod; required reviewers must approve before it runs and before it can read prod secrets [27] [34].
- Enable prevent self-review so the author cannot approve their own apply [27].
- On Free, Pro and Team plans required reviewers only work on public repositories; private infrastructure repositories need GitHub Enterprise Cloud [27].
- Use a wait timer if the change window requires a delay [27].

**Applying exactly the reviewed plan**

GenNoor Tech TRAINING INNOVATE BUILD
For more info - https://gennoor.com

- Upload the plan as an artifact with a short retention-days [33].
- A plan file contains backend config and possibly secrets in plain text, so protect artifacts and keep retention short [2] [19].
- Plans and applies on different machines need the same working directory, including .terraform, so archive it after plan [24].
- Record the plan's SHA-256 in the PR and verify it before apply.
- If main moved since the plan, re-plan after merge and require approval of the new plan.

**Instructor demo.**

1. Ask Claude Code to draft plan.yml and apply.yml from the course spec, then review the diff.
2. Open a pull request and show checks and the plan comment [25].
3. Merge and show the apply waiting for prod approval [27].
4. Approve as a second user and show the apply using the uploaded plan [33].
5. Trigger two merges quickly and show the concurrency queue [29].

**Hands-on lab.** Write the plan and apply workflows with Claude Code, open a pull request, read the plan comment and merge to see the gated apply run.

1. Draft plan.yml with Claude Code and review every line.
2. Add fmt, validate, tflint and plan steps and the PR comment step.
3. Draft apply.yml with environment: prod and a concurrency group.
4. Add the plan artifact upload and download with a hash check.
5. Make the checks required in branch protection.
6. Open a pull request that adds a virtual server and read the plan comment.
7. Merge, wait for approval and approve as your partner.
8. Verify the applied change on the lab F5.

**Lab success criteria.**

- Every pull request gets a plan comment and required checks.
- Apply runs only after merge and environment approval by someone other than the author.
- The applied plan hash matches the reviewed plan hash.
- Two applies never run at the same time.

**Deliverable.** Working plan-on-PR and apply-on-merge pipelines.

**Common pitfalls.**

- cancel-in-progress: true on apply, killing a run mid-change [29].
- Re-planning in the apply job instead of using the reviewed plan.
- Plan comments over the size limit failing silently [25].
- Assuming required reviewers work on a private repository on a Team plan [27].

**Knowledge check.**

Gennoor Tech
For more info - https://gennoor.com

1. How many required reviewers must approve an environment job? Answer: One of the listed reviewers (up to six can be listed) [27].
2. Why archive the working directory with the plan? Answer: Apply on another machine needs the same files, including .terraform [24].
3. What permission does the plan job need to comment on a PR? Answer: pull-requests: write [31].

**Slides and assets to prepare.**

- plan.yml and apply.yml reference workflows.
- Plan comment script.
- Environment and approval setup guide.
- Pipeline sequence diagram.

**Sources for this module.** 2, 4, 5, 7, 19, 24, 25, 26, 27, 29, 31, 33, 34

## Module 7: Guardrails, approvals and audit for Terraform and GitHub automation

**Goal.** Add policy checks, human approval and a per-change audit record so no unsafe plan reaches a device.

**Outline topics.**

- Policy checks before apply: allowed resource types, no destroy without a linked change ticket, no changes outside the PR scope
- GitHub environment protection rules and required reviewers as the human approval gate for every apply
- Audit record per change: who asked, what Claude recommended, plan hash, approver, apply result and run URL
- Least privilege for pipelines and agents: read-only device accounts for plan, scoped tokens, no long-lived credentials in the pipeline

**Learning objectives.**

- Learners can write policies over plan JSON that block disallowed resource types, unticketed destroys and out-of-scope changes.
- Learners can configure environment protection as the human approval gate.
- Learners can produce an audit record for each change.
- Learners can apply least privilege to pipelines and agents.

**Policy checks before apply**

- Write policies against terraform show -json: resource_changes[].change.actions contains delete means a destroy or replace [11] [23].
- Conftest runs Rego policies from policy/ against the plan JSON [36].
- Rule 1: only allowed types (bigip_ltm_*, citrixadc_*, aci_*) may change.
- Rule 2: any delete requires a CHG ticket in the PR title or body.

- Rule 3: resource addresses must be under the module folders touched by the PR.

**Environment approvals**

- Required reviewers on the prod environment approve each apply; prevent self-review is on [27].
- Environment secrets stay locked until approval, so an unapproved job cannot reach the device [34].
- Approvers see the plan comment, the policy result and Claude's verdict before approving.
- Branch protection plus environment protection gives two human checks: review and apply approval [5].

**Audit record**

- Fields: change ticket, requester, PR number, commit SHA, plan SHA-256, policy result, Claude verdict JSON, approver, approval time, apply result and run URL.
- Write the record as a JSON artifact and to an append-only store.
- PostToolUse hooks log every tool call made by the review agent [53].
- Never store plan contents with secrets in the audit store [19].

**Least privilege**

- Default permissions: {} at workflow level, then grant per job: contents: read, pull-requests: write, checks: write, id-token: write only where needed [31].
- GITHUB_TOKEN lasts only for the job and is limited to the repository [32].
- Plan uses read-only device accounts; apply uses write accounts fetched at run time.
- The review agent has no write credentials of any kind.

**Instructor demo.**

1. Show a Rego policy that denies delete without a ticket and run conftest against a plan [36].
2. Open a PR that destroys a pool without a ticket and show the policy failure.
3. Add the ticket and show the policy pass.
4. Show the prod approval screen with plan, policy and verdict links [27].
5. Export the audit record JSON for the change.

**Hands-on lab.** Add a policy job that blocks a destroy without a ticket, require approval on the prod environment, and export the audit record for one change.

1. Write policy/deny_delete_without_ticket.rego and policy/allowed_types.rego.
2. Add a policy job running conftest on the plan JSON [36].
3. Make the policy check required.
4. Turn on required reviewers and prevent self-review for prod [27].
5. Set workflow-level permissions: {} and per-job permissions [31].
6. Raise a PR that removes a pool without a ticket and observe the block.
7. Add the ticket, merge, approve and apply.

GennoorTech
for more info: https://gennoor.com

8. Export and review the audit record.

**Lab success criteria.**

- Unticketed destroys cannot merge.
- Prod apply cannot run without another person's approval.
- The audit record includes plan hash, verdict, approver and run URL.
- No job has more permissions than it needs.

**Deliverable.** Guardrail policy, approval gate and audit record format.

**Common pitfalls.**

- Checking only for ['delete'] and missing replacements, which include delete [11].
- Policies that read the HCL instead of the plan, missing module-generated changes.
- Granting write-all to every job for convenience [31].
- Storing full plan files in the audit store.

**Knowledge check.**

1. Which action lists in plan JSON mean the object will be deleted? Answer: ["delete"], ["delete","create"] and ["create","delete"] [11].
2. Which tool runs Rego policies against plan JSON in this course? Answer: Conftest [36].
3. What does permissions: {} do? Answer: Disables all GITHUB_TOKEN permissions for that scope [31].

**Slides and assets to prepare.**

- Rego policy pack.
- Audit record schema.
- Permissions matrix per job.
- Approval checklist for approvers.

**Sources for this module.** 5, 11, 19, 23, 27, 31, 32, 34, 36, 53

## Day 3

### Module 8: Claude as plan reviewer: Agent SDK, MCP and structured verdicts

**Goal.** Build an Agent SDK application that reads plan JSON, returns a structured risk verdict, posts it to the pull request and never merges or applies.

**Outline topics.**

- An Agent SDK application that reads terraform show -json and returns risk level, affected resources and a summary in a fixed JSON schema
- Posting the verdict as a pull request comment and as a check run through the GitHub API; the agent never merges or applies

GenNoorTech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

- Prompt and schema tests: deletes, replacements, certificate changes and out-of-scope resources must be flagged correctly
- Handling uncertainty: the agent marks unknowns and asks for a human when the plan touches resources it cannot classify

**Learning objectives.**

- Learners can define a verdict schema and get validated verdicts from the Agent SDK.
- Learners can post the verdict as a PR comment and a check run.
- Learners can test the reviewer against fixture plans for deletes, replacements, certificate changes and out-of-scope resources.
- Learners can make the agent mark unknowns and request a human.

**The verdict schema**

- Fields: risk (LOW, MEDIUM, HIGH, UNKNOWN), summary, findings[{address, action, reason, evidence_path}], requires_ticket, unclassified[], recommendation.
- Pass it as output_format json_schema from a Pydantic model and read structured_output [52].
- Treat error_max_structured_output_retries as UNKNOWN risk and fail the check [52].
- A deterministic pre-pass computes counts and replace_paths from plan JSON so Claude explains facts rather than discovering them [11].

**Running in the pull request workflow**

- The review job downloads the plan artifact, runs terraform show -json, redacts sensitive values and runs the agent [23] [33].
- Post a PR comment, and a check run with conclusion action_required or failure for HIGH risk [30].
- In Actions, GITHUB_TOKEN is a GitHub App installation token, so checks: write lets the job create check runs [32] [31].
- Keep annotations under 50 per API call [30].
- The agent has tools=[] and only a read-only plan tool; it cannot merge, approve or apply [51].

**Prompt and schema tests**

- Fixtures: a pool member delete, a virtual server replacement due to a name change, a certificate content change, an aci_tenant change outside scope, and a no-op.
- Assert the risk level and that each finding's address exists in resource_changes [11].
- Run tests on every change to the prompt or schema.
- Compare the agent's findings with the policy job; they must not disagree on deletes.

**Handling uncertainty**

- Resources not in the classification table go into unclassified[] and force risk UNKNOWN.
- Values in after_unknown are reported as 'known after apply', not guessed [11].
- The agent asks for a named human reviewer in the comment when risk is UNKNOWN.

- An optional route uses anthropics/claude-code-action in automation mode with a prompt; the course prefers the SDK app for the fixed schema [9].

**Instructor demo.**

1. Show the verdict model and a fixture plan with a hidden replacement.
2. Run the agent locally and show HIGH risk with replace_paths cited [11].
3. Run the PR workflow and show the comment and failing check run [30].
4. Introduce a resource type not in the table and show UNKNOWN and a human request.
5. Show the agent's tool list to prove it cannot apply [51].

**Hands-on lab.** Build and test the plan-review agent, run it inside the pull request workflow and watch it flag a hidden resource replacement.

1. Write the Pydantic verdict model.
2. Write the deterministic pre-pass over plan JSON.
3. Write the agent with output_format and a read-only plan tool.
4. Write pytest cases for the five fixture plans.
5. Add the review job to the PR workflow with checks: write and pull-requests: write.
6. Open a PR that renames a virtual server without a moved block.
7. Read the verdict comment and check run.
8. Fix with a moved block and see the verdict drop to LOW.

**Lab success criteria.**

- All fixture tests pass.
- The hidden replacement is flagged HIGH with the right address.
- Unclassified resources produce UNKNOWN and a human request.
- The agent has no merge, approve or apply capability.

**Deliverable.** Plan-review agent with schema tests wired into the PR pipeline.

**Common pitfalls.**

- Letting the model read raw plan text instead of JSON.
- Trusting structured_output without a success check [52].
- Using a personal token and expecting to create check runs; only GitHub Apps can [30].
- No regression tests when the prompt changes.

**Knowledge check.**

1. Which plan JSON field explains why a resource will be replaced? Answer: replace_paths [11].
2. Why can an Actions job create a check run with GITHUB_TOKEN? Answer: GITHUB_TOKEN is a GitHub App installation token, and checks: write grants the permission [32] [31].
3. What should the agent do with a resource it cannot classify? Answer: List it as unclassified, set risk UNKNOWN and request a human.

Gennoor Tech
AI · INNOVATE · BUILD
For more info - https://gennoor.com

**Slides and assets to prepare.**

- Verdict schema and sample verdicts.
- Fixture plan pack.
- Review job workflow snippet.
- Risk classification table for F5, NetScaler and ACI resources.

**Sources for this module.** 9, 11, 23, 30, 31, 32, 33, 51, 52

## Module 9: Terraform for F5, NetScaler, Cisco ACI and other load balancers

**Goal.** Use vendor providers for F5, NetScaler and ACI, and bring Radware Alteon and other load balancers into the same gated workflow.

**Outline topics.**

- F5 BIG-IP provider: bigip_ltm_node, bigip_ltm_pool, bigip_ltm_virtual_server and bigip_ssl_certificate backed by iControl REST
- Citrix ADC provider for NetScaler: citrixadc_lbvserver, citrixadc_servicegroup and bindings over the NITRO API
- Cisco ACI provider: aci_tenant, aci_application_profile, aci_bridge_domain against the APIC REST API
- Where no provider exists: Python wrappers for Radware Alteon and other load balancers (Anaya LB) driven from the same pipeline with the same approval gate

**Learning objectives.**

- Learners can manage F5 nodes, pools, virtual servers and certificates with the BIG-IP provider.
- Learners can manage NetScaler lbvservers, service groups and bindings with the citrixadc provider.
- Learners can manage ACI tenants, application profiles and bridge domains with the ACI provider.
- Learners can drive a Python change for Radware Alteon or another load balancer through the same approval gate.

**F5 BIG-IP provider**

- Resources: bigip_ltm_node, bigip_ltm_pool, bigip_ltm_pool_attachment, bigip_ltm_virtual_server and bigip_ssl_certificate, all over iControl REST [38] [39] [40].
- Pool names use full paths such as /Common/web-pool; members are address:port [40].
- bigip_ssl_certificate takes name, content (usually file()) and partition.
- For declarative app deployments, bigip_as3 with tenant_filter is an alternative; it needs AS3 3.10 or later and Administrator role [8].
- Current provider release is 1.29.0; pin ~> 1.29 [47].

**Citrix ADC provider for NetScaler**

- citrixadc_lbvserver with name, servicetype, ipv46, port and lbmethod (default LEASTCONNECTION) [41].
- citrixadc_servicegroup and citrixadc_lbvserver_servicegroup_binding with name and servicegroupname [42].
- Import bindings with the composite ID vserver,servicegroup [42].
- Provider endpoint and credentials come from the environment or variables, never literals.

**Cisco ACI provider**

- aci_tenant maps to fvTenant, DN uni/tn-{name} [43].
- aci_application_profile maps to fvAp (uni/tn-{name}/ap-{name}) and aci_bridge_domain to fvBD (uni/tn-{name}/BD-{name}), both with parent_dn [45] [44].
- Authenticate with password or private_key plus cert_name; certificate login suits pipelines [46].
- Import existing objects by DN, for example terraform import aci_tenant.example uni/tn-pay [43].

**Where no provider is used**

- Radware's alteon-sdk gives Python access to Alteon over REST; minimum Alteon 31.0.10.0 or 32.2.2.0 [55].
- Wrap Python changes as plan and apply steps: plan prints intended changes as JSON, apply runs only in the approved apply job.
- Anaya LB and other platforms use the same wrapper contract; if no API exists, the change stays manual with a ticket.
- The Claude reviewer reads the wrapper's plan JSON with the same schema as Terraform changes.
- Same environment gate, same audit record.

**Instructor demo.**

1. Plan an F5 virtual server with pool and certificate in the lab [39].
2. Plan a NetScaler lbvserver and service group binding [41] [42].
3. Plan an ACI tenant, application profile and bridge domain against the sandbox [43] [44] [56].
4. Run the Alteon wrapper's plan mode and show its JSON [55].
5. Show all four changes in one PR with one approval.

**Hands-on lab.** Add a NetScaler virtual server and an ACI bridge domain through the pull request workflow, and attach a Python-driven Radware change to the same gate.

1. Add modules/netscaler-vserver and call it from envs/dev.
2. Add modules/aci-bd with tenant, application profile and bridge domain.
3. Configure provider credentials from environment secrets.
4. Write the Alteon wrapper plan and apply commands using alteon-sdk or the recorded mock.
5. Add the wrapper to the plan and apply workflows.
6. Open one PR with all changes and read the plan comment and verdict.

Gemoor Tech TRAINING INNOVATE BUILD For more info - https://gemoor.com

7. Approve and apply to dev.
8. Import one pre-existing ACI tenant and confirm a clean plan.

**Lab success criteria.**

- NetScaler and ACI objects are created through the pipeline only.
- The Alteon change is gated by the same environment approval.
- The imported tenant shows no changes after import.
- Provider versions are pinned and locked.

**Deliverable.** Multi-platform Terraform modules behind one approval workflow.

**Common pitfalls.**

- Using short names where the F5 provider expects full paths [40].
- Forgetting the composite import ID for NetScaler bindings [42].
- Running the Alteon wrapper outside the gated job.
- Using the APIC admin password in the pipeline instead of certificate auth [46].

**Knowledge check.**

1. What is the DN format of an ACI bridge domain? Answer: uni/tn-{name}/BD-{name} [44].
2. What import ID does citrixadc_lbvserver_servicegroup_binding use? Answer: vserver,servicegroup [42].
3. What account role does bigip_as3 need? Answer: Administrator [8].

**Slides and assets to prepare.**

- Module reference solutions for F5, NetScaler and ACI.
- Alteon wrapper skeleton with plan and apply modes.
- Provider version matrix.
- Import ID cheat sheet.

**Sources for this module.** 8, 38, 39, 40, 41, 42, 43, 44, 46, 47, 55, 56, 45

## Module 10: Capstone: end-to-end change through the Claude-reviewed pipeline

**Goal.** Take a change request through pull request, Claude verdict, policy, approval, gated apply, audit and rollback, and hand the repository over to the team.

**Outline topics.**

- A change request in, a reviewed pull request, a Claude verdict, a human approval, a gated apply and an audit record out
- Runbook: how to raise, review, approve, apply and revert a change in this repository
- Handover: CODEOWNERS, environment reviewers and audit export for the team

Genoor Tech TRAINING INNOVATE BUILD For more info - https://genoor.com

- Acceptance review: mistakes caught, approvals enforced, rollback by reverting the pull request, demo to the group

**Learning objectives.**

- Learners can run a full change from ticket to applied change with audit record.
- Learners can write a runbook for raising, reviewing, approving, applying and reverting.
- Learners can hand over CODEOWNERS, environment reviewers and audit export.
- Learners can roll back by reverting a pull request and show the result.

**The capstone change**

- Ticket: add vs-api-443 on the lab F5 with a pool and certificate, plus an ACI bridge domain and a NetScaler failover vserver.
- One instructor-seeded problem: a rename that would replace a live virtual server.
- Teams must catch it through the verdict or policy and fix it with a moved block [12].
- Every step produces an audit entry.

**Runbook**

- Raise: branch, PR template, ticket link.
- Review: plan comment, policy, Claude verdict, CODEOWNERS approval [6].
- Approve: environment reviewer, not the author [27].
- Apply: saved plan only; Revert: git revert the merge commit, new PR, same gate.

**Handover and acceptance**

- CODEOWNERS and environment reviewers set to the real team.
- Audit export location and format documented.
- Acceptance: mistakes caught, approvals enforced, rollback demonstrated.
- Demo to the group with the audit record on screen.

**Instructor demo.**

1. Brief the change ticket and acceptance checklist.
2. Show a reference run of a small change end to end.
3. Show a revert PR and its plan.
4. Model a good approval decision using the verdict.

**Hands-on lab.** Capstone build and demo.

1. Create the change branch and modules for the ticket.
2. Open the PR and resolve checks.
3. Handle the seeded replacement finding.
4. Get CODEOWNERS review and merge.
5. Get environment approval and apply.
6. Export the audit record.

7. Revert the change through a PR and apply the revert.
8. Present the runbook and demo.

**Lab success criteria.**

- The seeded replacement is caught before merge.
- Apply happened only after another person's approval.
- The audit record is complete for both the change and the revert.
- The runbook is usable by someone who did not attend.

**Deliverable.** End-to-end Terraform and GitHub workflow with Claude review and runbook.

**Common pitfalls.**

- Approving your own change.
- Reverting by editing the device instead of reverting the PR.
- Skipping the audit export.

**Knowledge check.**

1. How is a change rolled back in this workflow? Answer: Revert the merge commit in a new PR, which goes through the same plan, review and approval.
2. Who may approve the prod apply? Answer: A required reviewer other than the person who started the deployment [27].
3. What fixes a rename that would replace a live virtual server? Answer: A moved block from the old to the new address [12].

**Slides and assets to prepare.**

- Capstone ticket and acceptance checklist.
- Runbook template.
- Audit record example.
- Demo script.

**Sources for this module.** 6, 12, 27

## 6. Capstone plan

A team GitHub repository where Terraform changes for F5, NetScaler, Cisco ACI and other load balancers (Anaya LB) go through a pull request, a Claude plan review, policy checks and a human approval before a gated apply, with an audit record for every change and rollback by revert.

Teams run a change for F5, NetScaler, Cisco ACI and a wrapper-driven load balancer through a pull request, a Claude plan review, policy checks and a human approval before a gated apply, with an audit record for every change and rollback by revert.

### Milestones

Genpoor Tech TRAIN. INNOVATE. BUILD. For more info - https://genpoor.com

1. Change branch and PR opened with ticket link.
2. Checks green: fmt, validate, tflint, policy and plan.
3. Seeded replacement caught and fixed with a moved block.
4. Code owner review and merge.
5. Environment approval and gated apply from the saved plan.
6. Audit record exported.
7. Revert PR applied through the same gate.

### Evaluation notes

- Fail if any apply ran without another person's approval.
- Check plan hash continuity from PR to apply.
- Check that the verdict and policy results agree on deletes and replacements.
- Assess runbook clarity for a new team member.
- Assess repository hygiene: no state, plans or secrets committed.

### Assessment weights

| Assessment area | Weight | Evidence |
|---|---|---|
| Working functionality | 25% | Demonstrable end-to-end workflow in a repeatable lab |
| Security and safety controls | 20% | Threat model, authorization boundaries, approval gates, safe failure |
| Testing and evaluation | 20% | Automated tests, golden dataset, measured quality and known limitations |
| Engineering quality | 15% | Clear interfaces, error handling, reproducible setup, maintainable code |
| Evidence and explainability | 10% | Traceable findings, sources, assumptions, logs |
| Documentation and presentation | 10% | Architecture diagram, README, runbook, demo, lessons learned |

## 7. Promotional video: research behind the script

**Opening scenario.** The scenario (an apply removes a production virtual server because nobody read 'minus one' in the plan at two in the morning) is a teaching scenario. It is grounded in documented behavior: plans mark destroys with - and replacements with -/+, and a renamed resource is planned as destroy and create unless a moved block is used [10] [12].

Gennoor Tech TRAIN · INNOVATE · BUILD For more info - https://gennoor.com

| Claim or on-screen text | Basis |
| --- | --- |
| 10 modules, 3 days, hands-on | Matches content.py: 10 modules over 3 days. |
| 5 common mistakes reproduced and fixed | Course design: unpinned providers, hard-coded secrets, committed state, count versus for_each, targeted apply drift (plus renames), each documented by HashiCorp [10] [13] [16] [19] [1]. |
| 0 applies without human approval | Enforced by GitHub environment required reviewers with prevent self-review; on Free, Pro and Team plans this requires public repositories, so private use needs Enterprise Cloud [27]. |
| Initializing provider F5Networks/bigip v1.22.2 … locked in .terraform.lock.hcl | Lock file behavior is accurate [16]. The version is illustrative: the registry lists 1.29.0 as current on the verification date, so update the on-screen version or treat it as an older pinned example [47]. |
| Plan: 3 to add, 0 to change, 0 to destroy | Standard plan summary format; numbers are scenario values [10]. |
| claude 'explain plan.tfplan' … Apply is not available to me | Design behavior from deny rules and a tool set without apply [48] [51]; the command line shown is illustrative of the course MCP tooling, which explains the plan through show -json [23]. |
| count = length(var.members) replaced by for_each = var.members on bigip_ltm_pool_attachment | Matches HashiCorp guidance on count versus for_each and the F5 pool attachment resource [13] [14] [40]. |
| password = var.bigip_password # from GitHub environment secret | Environment secrets released after approval and sensitive variables are documented; note the value still lands in state if passed to a resource [34] [18] [19]. |
| .gitignore: *.tfstate, *.tfplan, .terraform/ | The GitHub Terraform template covers state and .terraform/ but leaves plan files commented out, so *.tfplan must be added, as shown [35]. |
| gh pr create; checks fmt, tflint, policy pass; plan comment posted; environment prod waiting for approval | Consistent with fmt -check, TFLint, Conftest, the setup-terraform PR comment pattern and environment approvals [4] [26] [36] [25] [27]. Ticket CHG0041233 and repository names are fictional. |
| PLAN REVIEW risk HIGH: bigip_ltm_virtual_server.vs_api_443 will be REPLACED (name changed) | Replacement is visible in plan JSON actions and replace_paths, and renames plan as destroy and create [11] [12]. Values are fictional. |
| "An M C P server that exposes the Terraform C L I as tools, so the agent can run init, validate and plan, but never apply" | Course design using MCP tools and Claude Code permissions [50] [54] [48]. The course tool set is |

GenmoorTech TRAINING For more info: https://genmoor.com

| | |
|---|---|
| | fmt, validate, plan and show -json; init runs in the plan tool's working directory setup. |
| "Python wrappers for Radware Alteon and other load balancers (Anaya LB) where no provider exists" | Radware publishes alteon-sdk for Python REST access [55]. No public Anaya LB API documentation was found; the course uses a mock API for it, which should be stated to buyers. |
| "Nothing you do in the lab can reach a production device" | Lab design commitment (isolated lab network, trial and sandbox devices) [56] [57] [58]. |

## References


**[1]** State | Terraform | HashiCorp Developer. HashiCorp. https://developer.hashicorp.com/terraform/language/state
Supports: Module 1 and 3: what state is and where it should live. Terraform uses workspace state to map real resources to configuration; state is a JSON file that should not be edited directly or stored in version control, and HashiCorp recommends HCP Terraform or a remote backend.

**[2]** Backend block configuration overview. HashiCorp. https://developer.hashicorp.com/terraform/language/backend
Supports: Module 3: remote backends. A configuration can have only one backend block; Terraform writes backend configuration in plain text to .terraform/terraform.tfstate and to plan files, so credentials should come from environment variables.

**[3]** Modules overview. HashiCorp. https://developer.hashicorp.com/terraform/language/modules
Supports: Module 1 and 3: reusable modules. A module is a collection of resources managed together; modules can come from local paths, a registry or VCS, and registry modules download automatically when source and version are set.

**[4]** terraform fmt command reference. HashiCorp. https://developer.hashicorp.com/terraform/cli/commands/fmt
Supports: Module 4 and 6: formatting gate. terraform fmt rewrites files to one canonical style with no options; -check returns non-zero when files are not formatted and -recursive processes subdirectories.

**[5]** About protected branches. GitHub Docs. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
Supports: Module 5: branch protection. Rules can require status checks or linear history, block force pushes and deletion by default, and do not apply to admins unless 'Do not allow bypassing the above settings' is enabled; only one branch protection rule applies to a branch at a time, unlike rulesets.

**[6]** About code owners. GitHub Docs. https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-code-owners
Supports: Module 5 and 10: CODEOWNERS. CODEOWNERS in .github/, the root or docs/ requests reviews automatically when a pull request touches owned files; the base branch copy is used, owners need write access, invalid lines are skipped and files over 3 MB are not loaded.

**[7]** Automate Terraform with GitHub Actions. HashiCorp. https://developer.hashicorp.com/terraform/tutorials/automation/github-actions
Supports: Module 6: reference plan-on-PR, apply-on-merge workflow. The tutorial runs a plan for every commit to a pull request branch and applies on push to main (including merges); it authenticates to HCP Terraform with a team token stored as the TF_API_TOKEN secret.

**[8]** AS3 Integration with Terraform. F5 (clouddocs.f5.com). https://clouddocs.f5.com/products/orchestration/terraform/latest/userguide/as3-integration.html

Supports: Module 9: declarative F5 option. bigip_as3 takes a required as3_json declaration and optional tenant_filter; it needs BIG-IP 12.1.x or later, AS3 3.10 or later and an Administrator-role account.

**[9]** Claude Code GitHub Actions. Anthropic (code.claude.com). https://code.claude.com/docs/en/github-actions
Supports: Module 8: alternative way to run Claude in a workflow. anthropics/claude-code-action runs in interactive mode on @claude mentions when no prompt is given, or automation mode on any event when a prompt is supplied; it uses an ANTHROPIC_API_KEY or CLAUDE_CODE_OAUTH_TOKEN secret, requires write access from the triggering user and rejects bots unless listed in allowed_bots.

**[10]** terraform plan command reference. HashiCorp.
https://developer.hashicorp.com/terraform/cli/commands/plan
Supports: Module 3, 4 and 6: plan symbols, saved plans, targeting and drift checks. Plan symbols are + create, - destroy, ~ update in place and -/+ destroy then create replacement; -out=FILE saves a plan to apply later, mainly for automation; -detailed-exitcode returns 0 for no changes, 1 for error and 2 for changes; -target is for exceptional circumstances only; -refresh-only updates state to match remote objects.

**[11]** JSON Output Format. HashiCorp. https://developer.hashicorp.com/terraform/internals/json-format
Supports: Module 8: what the plan-review agent reads. Plan JSON has resource_changes[], each with address and change.actions, which is one of ["no-op"], ["create"], ["read"], ["update"], ["delete","create"], ["create","delete"] or ["delete"]; before, after and after_unknown give values; before_sensitive and after_sensitive mark sensitive leaves; replace_paths lists the attributes that forced replacement.

**[12]** Refactor modules (moved block). HashiCorp.
https://developer.hashicorp.com/terraform/language/modules/develop/refactoring
Supports: Module 4: renames without destroy. By default Terraform treats a changed address as destroy and create; a moved { from = ... to = ... } block renames the object in state so it is not destroyed, and also maps count instances to the new address.

**[13]** The count meta-argument. HashiCorp. https://developer.hashicorp.com/terraform/language/meta-arguments/count
Supports: Module 4: count versus for_each. Use count for nearly identical instances and for_each when instances need distinct values that cannot be derived from an integer index; count and for_each cannot be used together; count must be known before apply; instances are addressed as name[0].

**[14]** The for_each meta-argument. HashiCorp. https://developer.hashicorp.com/terraform/language/meta-arguments/for_each
Supports: Module 4 and 9: keyed instances. for_each takes a map or set of strings, exposing each.key and each.value; keys must be known values, otherwise Terraform reports that for_each has dependencies that cannot be determined before apply.

**[15]** Version constraints. HashiCorp. https://developer.hashicorp.com/terraform/language/expressions/version-constraints
Supports: Module 3 and 4: pinning providers. ~> allows only the right-most version component to increment: ~> 1.0.4 allows 1.0.5 and 1.0.10 but not 1.1.0; ~> 1.1 allows 1.2 and 1.10 but not 2.0.

**[16]** Dependency Lock File (.terraform.lock.hcl). HashiCorp.
https://developer.hashicorp.com/terraform/language/files/dependency-lock
Supports: Module 3 and 4: lock file in Git. terraform init creates or updates .terraform.lock.hcl; it should be committed so dependency changes are code reviewed; Terraform re-selects the locked version unless terraform init -upgrade is run.

**[17]** terraform import command reference. HashiCorp.
https://developer.hashicorp.com/terraform/cli/commands/import
Supports: Module 3: bringing existing objects under management. Usage is terraform import [options] ADDRESS ID; as an alternative, an import block in configuration lets terraform apply import resources, which suits CI/CD.

**[18]** Use input variables. HashiCorp. https://developer.hashicorp.com/terraform/language/values/variables
Supports: Module 3 and 4: variables and secrets. Set sensitive = true on secret variables to hide them in CLI output; environment variables named TF_VAR_<name> set input variables, useful in CI/CD.

**[19]** Manage sensitive data. HashiCorp. https://developer.hashicorp.com/terraform/language/manage-sensitive-data
Supports: Module 4 and 7: why secrets in config end up in state. If you add secret values directly to configuration, Terraform stores them in state and plan files; state and plan files contain detailed infrastructure information that may include sensitive values.

**[20]** State: Locking. HashiCorp. https://developer.hashicorp.com/terraform/language/state/locking
Supports: Module 3 and 6: state locking. If the backend supports it, Terraform locks state automatically for all operations that could write state; not all backends support locking; force-unlock should be used only to release your own lock when automatic unlocking failed; the documentation version selector lists v1.17.x as latest at verification.

**[21]** Managing workspaces. HashiCorp. https://developer.hashicorp.com/terraform/cli/workspaces
Supports: Module 3: workspaces versus separate directories. Every initialized directory starts with a default workspace, managed with terraform workspace list/new/delete; HashiCorp recommends other approaches for complex deployments that need separate credentials and access controls.

**[22]** terraform state mv command reference. HashiCorp.
https://developer.hashicorp.com/terraform/cli/commands/state/mv
Supports: Module 3: moving state. terraform state mv [options] SOURCE DESTINATION retains an existing remote object but tracks it at a different address; the moved block is the configuration-based alternative.

**[23]** terraform show command reference. HashiCorp.
https://developer.hashicorp.com/terraform/cli/commands/show
Supports: Module 2 and 8: JSON plan output. terraform show -json on a plan file prints the plan, configuration and current state as JSON; with -json, sensitive values in state are displayed in plain text.

**[24]** Running Terraform in automation. HashiCorp.
https://developer.hashicorp.com/terraform/tutorials/automation/automate-terraform
Supports: Module 6: non-interactive plan and apply. Use terraform init -input=false, terraform plan -out=tfplan -input=false and terraform apply -input=false tfplan; TF_IN_AUTOMATION adjusts output for wrappers; when plan and apply run on different machines, archive the whole working directory including .terraform after plan.

**[25]** hashicorp/setup-terraform. HashiCorp (GitHub). https://github.com/hashicorp/setup-terraform
Supports: Module 6: installing Terraform in Actions. Inputs include terraform_version, cli_config_credentials_token and terraform_wrapper (default true); the wrapper exposes stdout, stderr and exitcode outputs; the README shows posting plan output as a PR comment with actions/github-script and warns GitHub comments are limited to 65535 characters; examples use hashicorp/setup-terraform@v4.

**[26]** TFLint. terraform-linters (GitHub). https://github.com/terraform-linters/tflint
Supports: Module 4: linting. TFLint is a pluggable linter that flags possible errors, deprecated syntax and unused declarations and enforces best practices; it is configured in .tflint.hcl, plugins install with tflint --init, the Terraform language ruleset is bundled, output formats include json, junit and sarif.

**[27]** Deployments and environments. GitHub Docs. https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments
Supports: Module 6 and 7: environment approvals. Required reviewers (up to six users or teams, one approval needed) gate jobs that reference an environment; prevent self-review stops the initiator from approving; wait timers range 1 to 43,200 minutes; on Free, Pro and Team plans required reviewers are only available for public repositories.

**[28]** OpenID Connect. GitHub Docs. https://docs.github.com/en/actions/concepts/security/openid-connect
Supports: Module 5 and 7: short-lived cloud credentials. With OIDC you do not store cloud credentials as long-lived GitHub secrets; the workflow requests a short-lived access token from the cloud provider through a configured trust.

**[29]** Control the concurrency of workflows and jobs. GitHub Docs. https://docs.github.com/en/actions/how-tos/write-workflows/choose-when-workflows-run/control-workflow-concurrency

Supports: Module 6: no parallel applies. At most one job or workflow runs in a concurrency group; a new queued run makes the existing pending run cancel by default; cancel-in-progress: true also cancels the running one.

**[30]** REST API endpoints for check runs. GitHub Docs. https://docs.github.com/en/rest/checks/runs
Supports: Module 8: posting the verdict as a check run. Only GitHub Apps can create check runs; conclusion values include action_required, cancelled, failure, neutral, success, skipped and timed_out; at most 50 annotations per request; Actions steps are limited to 10 warning and 10 error annotations.

**[31]** Workflow syntax for GitHub Actions: permissions. GitHub Docs.
https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
Supports: Module 7 and 8: least-privilege GITHUB_TOKEN. The permissions key sets read, write or none per scope (for example checks, contents, id-token, pull-requests); unspecified scopes become none; checks: write permits creating a check run; permissions: {} disables all.

**[32]** GITHUB_TOKEN. GitHub Docs. https://docs.github.com/en/actions/concepts/security/github_token
Supports: Module 7 and 8: what the workflow token is. GITHUB_TOKEN is a GitHub App installation token limited to the repository, created per job and expiring when the job ends (at most 24 hours on self-hosted runners); events it triggers do not start new workflow runs except listed cases such as pull_request opened, which wait for approval.

**[33]** Store and share data with workflow artifacts. GitHub Docs.
https://docs.github.com/en/actions/tutorials/store-and-share-data
Supports: Module 6: carrying the reviewed plan to apply. actions/upload-artifact@v4 uploads files with an optional retention-days that cannot exceed the repository or organization limit; actions/download-artifact downloads them, and cross-run downloads need a token and run ID.

**[34]** Using secrets in GitHub Actions. GitHub Docs. https://docs.github.com/en/actions/concepts/security/secrets
Supports: Module 5 and 7: environment secrets. For environment secrets you can enable required reviewers; a job cannot access environment secrets until required approvers grant approval.

**[35]** github/gitignore: Terraform.gitignore. GitHub (github/gitignore).
https://raw.githubusercontent.com/github/gitignore/main/Terraform.gitignore
Supports: Module 4 and 5: what to keep out of Git. The template ignores .terraform/, *.tfstate, *.tfstate.*, crash logs, *.tfvars, *.tfvars.json, override files, .terraform.tfstate.lock.info and .terraformrc; ignoring tfplan files is left commented out, so *.tfplan must be added explicitly.

**[36]** Conftest. Open Policy Agent project. https://www.conftest.dev/
Supports: Module 7: policy checks on plan JSON. Conftest tests structured configuration data, including Terraform, with policies written in Rego; policies live in a policy directory by default, overridable with --policy.

**[37]** pre-commit. pre-commit project. https://pre-commit.com/
Supports: Module 4: local hooks. Hooks are declared in .pre-commit-config.yaml at the repository root and installed with pre-commit install, after which they run on every git commit; the same hooks can run in CI.

**[38]** F5 BIG-IP provider documentation (index). F5 Networks (GitHub).
https://raw.githubusercontent.com/F5Networks/terraform-provider-bigip/master/docs/index.md
Supports: Module 9: F5 provider configuration. The provider manages LTM, network and system objects and supports AS3/DO; address, username and password can be set with BIGIP_HOST, BIGIP_USER and BIGIP_PASSWORD; token_auth defaults to true.

**[39]** bigip_ltm_virtual_server resource documentation. F5 Networks (GitHub).
https://raw.githubusercontent.com/F5Networks/terraform-provider-bigip/master/docs/resources/bigip_ltm_virtual_server.md
Supports: Module 3 and 9: F5 virtual server resource. Required arguments are name, destination and port, with optional pool; an existing virtual server is imported by full path, for example terraform import bigip_ltm_virtual_server.http /Common/terraform_vs_http.

**[40]** bigip_ltm_pool_attachment resource documentation. F5 Networks (GitHub).
https://raw.githubusercontent.com/F5Networks/terraform-provider-bigip/master/docs/resources/bigip_ltm_pool_attachment.md

Supports: Module 4 and 9: pool members. pool (full path such as /Common/my-pool) and node (address or FQDN with port, for example 1.1.1.1:80) are required; imports take the pool full path and the node full path with port.

**[41]** citrixadc_lbvserver resource documentation. Citrix / NetScaler (GitHub).
https://raw.githubusercontent.com/citrix/terraform-provider-citrixadc/master/docs/resources/lbvserver.md
Supports: Module 9: NetScaler virtual server. Arguments include name, servicetype, ipv46, port and lbmethod (default LEASTCONNECTION); import with terraform import citrixadc_lbvserver.tf_lbvserver tf_lbvserver.

**[42]** citrixadc_lbvserver_servicegroup_binding resource documentation. Citrix / NetScaler (GitHub).
https://raw.githubusercontent.com/citrix/terraform-provider-citrixadc/master/docs/resources/lbvserver_servicegroup_binding.md
Supports: Module 9: binding a service group. name and servicegroupname are required; import uses the composite ID vserver,servicegroup, for example tf_lbvserver,tf_servicegroupname.

**[43]** aci_tenant resource documentation. Cisco DevNet (GitHub).
https://raw.githubusercontent.com/CiscoDevNet/terraform-provider-aci/master/docs/resources/tenant.md
Supports: Module 9: ACI tenant. aci_tenant maps to class fvTenant with DN format uni/tn-{name}; import with terraform import aci_tenant.example uni/tn-{name}.

**[44]** aci_bridge_domain resource documentation. Cisco DevNet (GitHub).
https://raw.githubusercontent.com/CiscoDevNet/terraform-provider-aci/master/docs/resources/bridge_domain.md
Supports: Module 9: ACI bridge domain. aci_bridge_domain maps to class fvBD with DN uni/tn-{name}/BD-{name} and a parent_dn argument; import by DN, for example terraform import aci_bridge_domain.example_tenant uni/tn-{name}/BD-{name}.

**[45]** aci_application_profile resource documentation. Cisco DevNet (GitHub).
https://raw.githubusercontent.com/CiscoDevNet/terraform-provider-aci/master/docs/resources/application_profile.md
Supports: Module 9: ACI application profile. aci_application_profile maps to class fvAp with DN uni/tn-{name}/ap-{name} and a parent_dn argument; import by DN.

**[46]** ACI provider documentation (index). Cisco DevNet (GitHub).
https://raw.githubusercontent.com/CiscoDevNet/terraform-provider-aci/master/docs/index.md
Supports: Module 9: ACI provider authentication. The provider takes url and either password or private_key with cert_name (certificate-based login).

**[47]** Terraform Registry API: F5Networks/bigip provider. HashiCorp (Terraform Registry).
https://registry.terraform.io/v1/providers/F5Networks/bigip
Supports: Lab environment and promo check: current provider versions. On 2026-10-30 the registry lists F5Networks/bigip 1.29.0 (published 2026-10-06); the same API lists citrix/citrixadc 3.0.3 and CiscoDevNet/aci 2.20.0.

**[48]** Configure permissions - Claude Code Docs. Anthropic (code.claude.com).
https://code.claude.com/docs/en/permissions
Supports: Module 2: keeping apply off the allowed list. Rules are evaluated deny, then ask, then allow; a broad deny such as Bash(aws *) blocks calls even if a narrower allow matches; Bash rules match the command text Claude writes and are not a security boundary around a program, so pair them with the sandbox or remove the tool when a restriction must hold.

**[49]** How Claude remembers your project - Claude Code Docs. Anthropic (code.claude.com).
https://code.claude.com/docs/en/memory
Supports: Module 2: CLAUDE.md. Project instructions live in ./CLAUDE.md or ./.claude/CLAUDE.md; /init generates a starting file; CLAUDE.md is context, not enforced configuration, so blocking needs a PreToolUse hook.

**[50]** Connect Claude Code to tools via MCP - Claude Code Docs. Anthropic (code.claude.com).
https://code.claude.com/docs/en/mcp
Supports: Module 2: registering the Terraform MCP server. claude mcp add [options] <name> -- <command>

[args...] adds a stdio server; --scope project stores it in .mcp.json for the team; project servers need approval on first use.

**[51]** Give Claude custom tools - Claude Agent SDK. Anthropic (code.claude.com). https://code.claude.com/docs/en/agent-sdk/custom-tools
Supports: Module 2 and 8: Terraform tools in the SDK. Use @tool and create_sdk_mcp_server; tools are named mcp__{server}__{tool}; tools=[] removes all built-ins; handlers can return is_error; annotations such as readOnlyHint are not enforcement.

**[52]** Get structured output from agents - Claude Agent SDK. Anthropic (code.claude.com). https://code.claude.com/docs/en/agent-sdk/structured-outputs
Supports: Module 8: the verdict schema. output_format={'type':'json_schema','schema':...} returns validated data in ResultMessage.structured_output; schemas are validated as draft-07; error_max_structured_output_retries means no valid output; success without structured_output is also a failure.

**[53]** Intercept and control agent behavior with hooks - Claude Agent SDK. Anthropic (code.claude.com). https://code.claude.com/docs/en/agent-sdk/hooks
Supports: Module 2 and 7: blocking and auditing tool calls. PreToolUse hooks can return permissionDecision 'deny' with a reason; matchers match tool names only; deny wins when hooks disagree; PostToolUse suits audit logging.

**[54]** Tools - Model Context Protocol specification (2025-06-18). Model Context Protocol. https://modelcontextprotocol.io/specification/2025-06-18/server/tools
Supports: Module 2: MCP tool contract. tools/list and tools/call; tools declare inputSchema and optional outputSchema; results carry content, structuredContent and isError; servers MUST validate inputs, enforce access controls, rate limit and sanitize outputs; clients SHOULD log tool use.

**[55]** alteon-sdk (PyPI project metadata). PyPI (package published by a Radware author). https://pypi.org/pypi/alteon-sdk/json
Supports: Module 9: Radware Alteon where no Terraform provider is used. alteon-sdk provides a Python API to Alteon structures via a REST backend (for example SlbNewCfgEnhRealServerTable beans read through AlteonClient); minimum supported Alteon versions are 31.0.10.0 and 32.2.2.0; the latest PyPI release is a beta (0.16b2).

**[56]** ACI Sandboxes - Cisco DevNet. Cisco DevNet. https://developer.cisco.com/docs/aci/sandbox
Supports: Lab environment: APIC for Terraform labs. DevNet offers an Always-On ACI Simulator (shared, no reservation) and a reservable ACI Simulator (up to 5 consecutive days) for REST API work.

**[57]** BIG-IP Virtual Edition free trial. F5. https://www.f5.com/trials/big-ip-virtual-edition
Supports: Lab environment: BIG-IP VE. F5 offers a 30-day free BIG-IP VE and BIG-IQ VE trial requested through MyF5.

**[58]** NetScaler VPX FAQ (13.1). NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/vpx/13-1/deploy-vpx-faq.html
Supports: Lab environment: NetScaler VPX Express. VPX Express is free with no license file and, in the newer version, no expiry; a retail license is needed for support cases.

For more info - https://gennoor.com